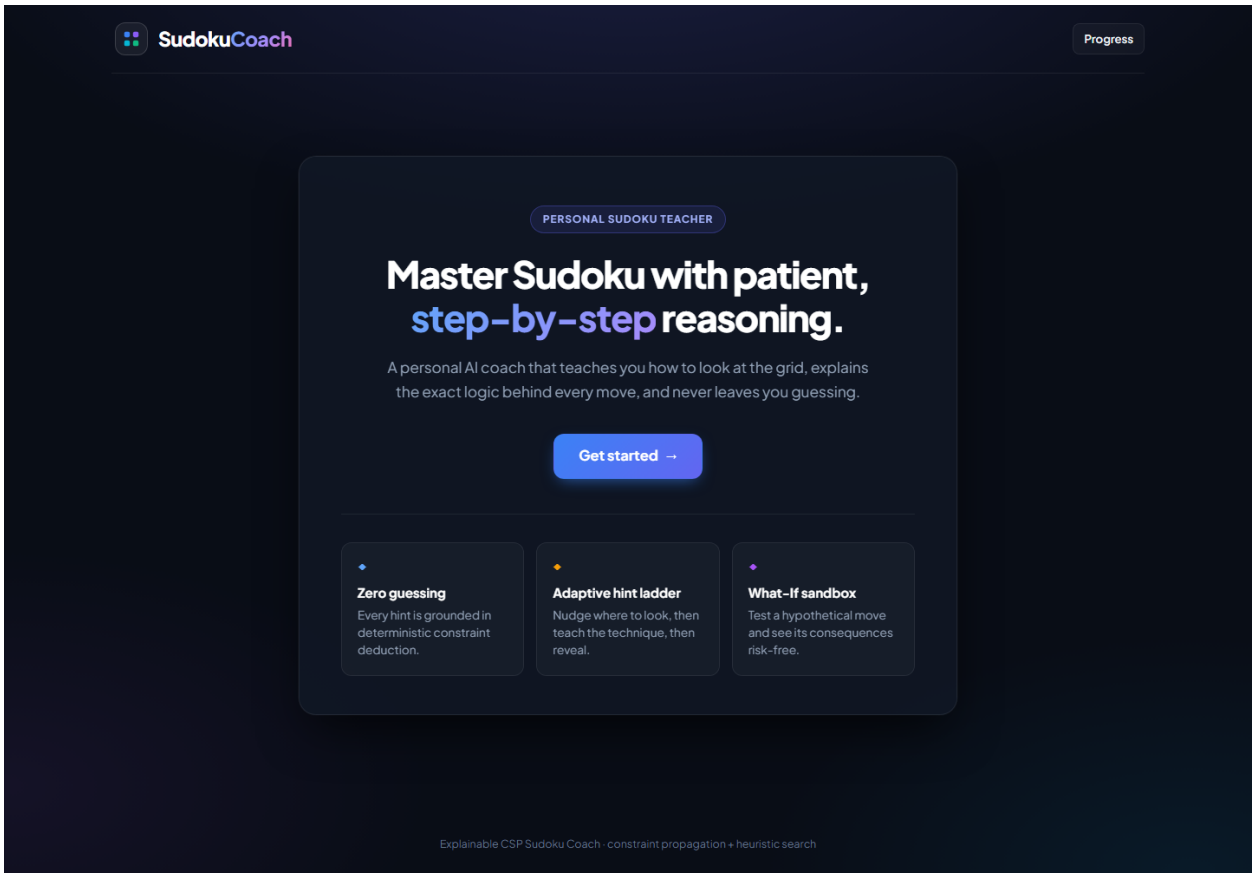
Sudoku AI Coach

How the project works, in plain words, plus common questions

Team review guide · CSE 311 Artificial Intelligence · October 2026



The home page at http://localhost:8000

How to use this guide. Read section 1 (the one-minute version) first. Sections 2–8 explain each part so anyone on the team can answer a question about it. Section 9 lists questions we are likely to be asked, with short answers.

Team and work division

Member	Owns	Presents
Sathwik Teja (lead) 2024BCD0040	Neuro-symbolic AI Teacher (LLM, claim verification, offline fallback), system architecture and FastAPI backend. Leads the Q&A, so knows every section. Focus: sections 4 and 8.	Slides 1–2, 8–9, 14–15
Sawan Rana 2024BCD0040	CSP search engine: Sudoku as a CSP, backtracking, forward checking, AC-3, MRV, LCV, Norvig propagation, and the unique-solution puzzle generator. Also limitations and future work. Focus: sections 5–6.	Slides 5–6, 13
Sunil 2024BCS0340	Rule base of 9 human techniques, hint ladder, move checker, What-If, and the test suite (48 tests). Focus: sections 6–7.	Slides 7, 10
Rishikesh 2024BCD0076	Web interface (grid, numpad, chat panel), lessons and skill check. Presents the introduction and team slides and runs the live demo. Focus: sections 1 and 3.	Slides 3–4, 11–12

Q&A is on slide 15: Sathwik leads, and each member answers questions about their own part, so everyone should read section 9. Each slide in the deck has a small "Presenter:" tag at the bottom right.

1. The one-minute version

The Sudoku AI Coach is a web app that **teaches** Sudoku instead of just solving it. Behind it is a **Constraint Satisfaction Problem (CSP) engine**: the same kind of AI we studied in class (forward checking, AC-3, MRV, LCV, backtracking search). We use that engine for three things:

- **Explain** every move in human terms ("the only place a 3 can go in Row B is…").
- **Check** the player's moves: is it a rule break, a legal move that leads to a dead end, or correct?
- **Simulate** "what if I put a 7 here?" on a copy of the board, without changing the real one.

On top of that there is an **AI Teacher chat**. It is *neuro-symbolic*: the CSP engine does all the reasoning and checking, and a language model (LLM) is optional, used only to phrase the answer nicely. Every digit the LLM claims is checked against the solver before it is shown. With no API key, the chat still works using the engine's own explanations, which is how we run it by default.

Key message for the evaluation

"The answers come from logic, not from a black box. Every hint can be traced back to a constraint rule, and the LLM is never trusted on facts. It only does the talking."

2. Running it

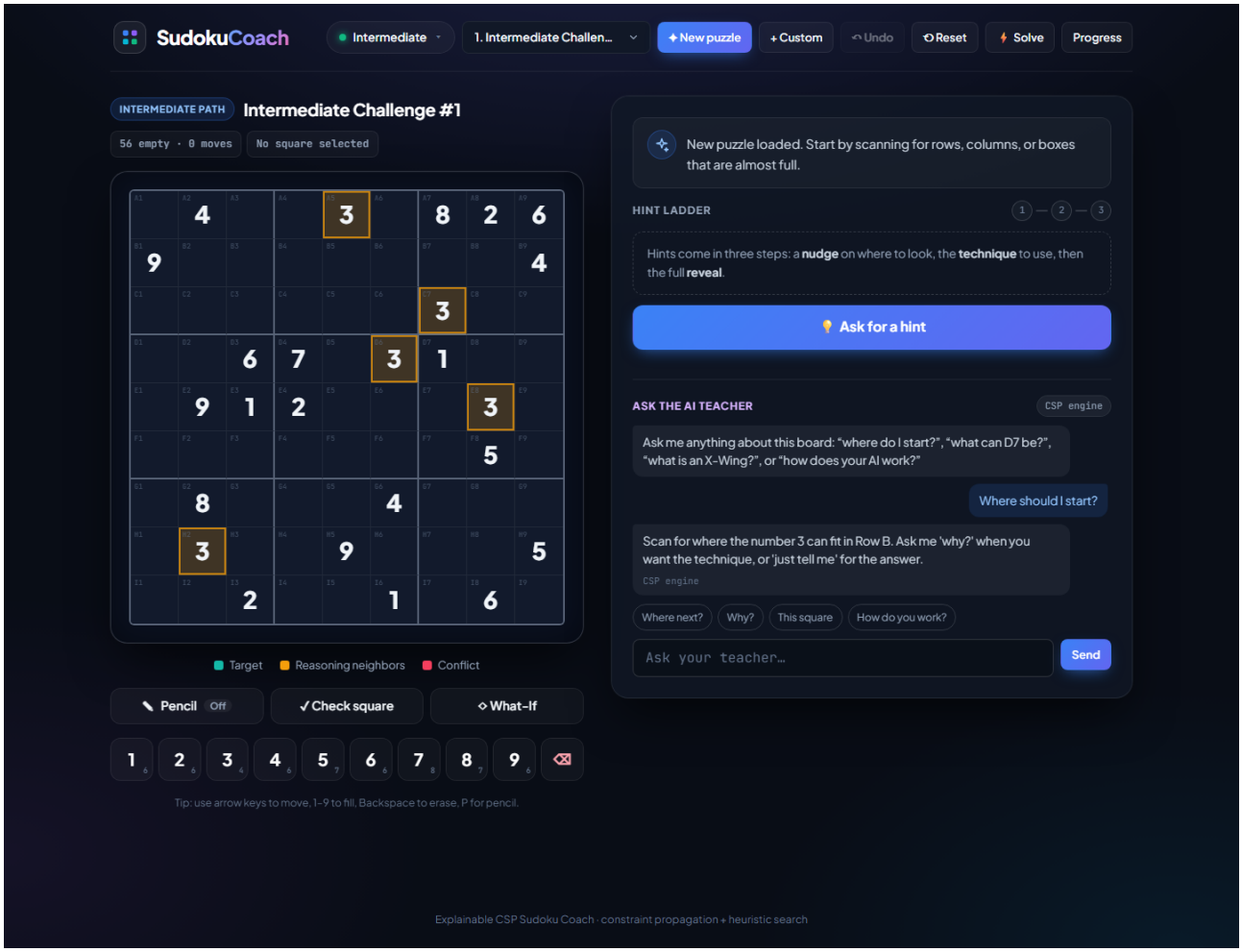
```
pip install fastapi uvicorn httpx
python -m uvicorn sudoku_ai_coach.api.app:app --host 127.0.0.1 --port 8000
# then open http://localhost:8000

# tests (48 in total, all passing)
python -m unittest discover -s tests -t .
python -m unittest discover -s sudoku_ai_coach/tests -t .
```

Optional: to let an LLM phrase the AI Teacher's answers, copy `.env.example` to `.env` and set `AI_TEACHER_API_KEY`. Any OpenAI-compatible provider works (OpenAI, Groq free tier, Gemini, OpenRouter). Never commit `.env`; it is already in `.gitignore`.

3. What the user sees

- **Home** → **choose a level** (Beginner / Intermediate / Advanced), or take a 4-question skill check that recommends one.
- **Beginner path:** 7 guided lessons on a real board: the one rule, reading the grid, Full House, Hidden Single, Naked Single, pencil marks, first full puzzle.
- **Puzzle path:** graded puzzles from the puzzle bank, or **New puzzle** to generate a fresh one for your level.
- **Board tools:** pencil marks, **Check square**, **What-If**, Undo, Reset, and Solve (the CSP solver finishes it).
- **Hint ladder:** 3 steps: a nudge (where to look) → the technique → the full reveal.
- **Ask the AI Teacher:** free-form chat about the current board, with quick buttons: Where next? / Why? / This square / How do you work?



Puzzle workspace: board on the left, hint ladder and AI Teacher on the right. The teacher answered "Where should I start?" with a nudge and highlighted the 3s.

4. Architecture

```
Browser  (static/index.html, app.js, style.css)
      |  JSON over HTTP
      v
FastAPI backend  (sudoku_ai_coach/api/app.py)
  |-- board_state.py       current board, move history, undo
  |-- techniques/          9 human techniques (Full House ... X-Wing)
  |-- hint_ladder.py       nudge -> technique -> reveal, mistake detection
  |-- move_checker.py      Case 1 / 2 / 3 move feedback
  |-- what_if.py           sandboxed simulation on a copy of the board
  |-- lesson_engine.py     7-lesson beginner curriculum
  |-- skill_check.py       4-question level placement
  |-- puzzle_generator.py  new, uniquely solvable, level-graded puzzles
  |-- ai_teacher.py        neuro-symbolic chat (CSP grounds + verifies, LLM phrases)
      |
      v
CSP engine  (src/)
  board.py       81 squares A1..I9, 27 units, 20 peers per square
  csp_solver.py  forward checking / AC-3 / Norvig propagation + MRV, LCV
  validator.py   checks a grid obeys every AllDifferent constraint
```

Cells use **Norvig notation**: rows A–I from top to bottom, columns 1–9 from left to right, so `D7` is row 4, column 7. Each cell has 20 **peers** (the other cells in its row, column and box) and belongs to 3 **units**.

5. The CSP engine (the "AI" from the course)

Concept	In Sudoku terms	Where
Variables	The 81 cells	src/board.py
Domains	Digits 1–9 still possible in a cell (its *candidates*)	src/board.py
Constraints	27 AllDifferent units: 9 rows, 9 columns, 9 boxes	src/board.py
Forward checking	After placing a digit, remove it from all 20 peers' domains	csp_solver.py
AC-3	Keep removing values until every pair of peers is consistent	csp_solver.py
Norvig propagation	Two rules repeated: (1) a cell with one candidate is fixed; (2) a digit with only one place in a unit goes there	csp_solver.py
MRV	Pick the cell with the fewest candidates next (fail-first)	csp_solver.py
LCV	Try the value that removes the fewest options from peers first	csp_solver.py
Backtracking	If a domain becomes empty, undo and try the next value	csp_solver.py

The coach uses the solver once per puzzle to get the **unique solution**. That solution is the ground truth for the move checker, the mistake detector and the AI Teacher's verifier.

6. Human techniques and puzzle grading

A computer can solve any puzzle by search, but a learner needs *human* steps. The technique engine (`api/techniques/`) looks for 9 techniques, always trying the easiest first:

Tier	Techniques
Beginner	Full House, Naked Single, Hidden Single
Intermediate	Naked Pair, Hidden Pair, Pointing Pair, Box/Line Reduction
Advanced	Naked Triple, X-Wing

Puzzle generator ("dig with a human solver"): build a random full grid, then blank cells one by one, keeping a removal only if the puzzle is still solvable using techniques at or below the target level. A puzzle that can be finished with forced deductions only has exactly **one** solution, so uniqueness comes for free. A puzzle is accepted only if it actually *needs* its level (an intermediate puzzle must not be solvable with singles alone). Advanced puzzles are rare, so a background pool keeps a few ready (`data/generated_pool.json`).

7. Hints, move checking and What-If

Hint ladder (3 steps)

- **1. Nudge:** where to look ("scan for where 3 can go in Row B").
- **2. Technique:** names and explains the technique that applies.
- **3. Reveal:** the cell, the digit and the full reason.
- If the board already has a **mistake**, the ladder points that out first, before any forward hint.

Move checker (Check square)

- **Case 1 – rule break:** the digit already appears in a peer; the conflicting cell is shown.
- **Case 2 – legal but wrong:** breaks no rule yet but differs from the unique solution, so it will lead to a dead end.
- **Case 3 – correct:** explains *why* (Naked Single, Hidden Single, or correct but not yet forced).

What-If sandbox

Places the digit on a **deep copy** of the board, runs constraint propagation and reports the chain of consequences, e.g. "F8 ends up with no candidates, so this move causes a contradiction". The real board is never changed.

8. The AI Teacher chat (ai_teacher.py)

Every message goes through six steps:

Step	What happens
1. Intent detection	Rule-based keyword matching decides what was asked: hint, reveal, a specific cell, a technique, an AI concept (CSP, MRV, AC-3…) or a mistake check.
2. Symbolic grounding	The CSP / technique engine runs on the live board and collects verified facts: candidates, the next deduction, mistakes, the unique solution.
3. Information gating	Only facts the learner is allowed to see are passed on. The answer digit is withheld unless they explicitly ask for it, so the tutor can't spoil the puzzle.
4. Explanation	If an API key is set, an LLM turns the facts into a friendly lesson (its system prompt says "use ONLY the facts provided"). Otherwise the engine's own template is used.
5. Verification	Every "cell = digit" claim in the reply is extracted with regular expressions and checked against the unique solution. One wrong claim rejects the whole reply.
6. Fallback	No key, a timeout (20 s) or a rejected reply → the engine's own explanation is shown, so the chat always works offline.

The small label under each reply shows where it came from: `CSP engine` or `llm, verified`. Endpoints: `POST /api/teacher/chat` and `GET /api/teacher/ai-status`.

9. Common questions and answers

Q. Is this really AI, or just a solver?

It is classical (symbolic) AI: a CSP with constraint propagation and heuristic search, which is exactly the course material. The new part is using that engine to *explain* its reasoning, check moves and simulate hypotheticals, not only to output a solution.

Q. Why not just use ChatGPT to teach Sudoku?

LLMs are bad at exact logic: they often state wrong digits confidently. In our design the LLM never decides anything. The CSP engine finds the facts, and any digit the LLM writes is checked against the solver before the user sees it. A wrong reply is thrown away and replaced by the engine's answer.

Q. Does it need internet or an API key?

No. Without a key the AI Teacher runs on the CSP engine alone (the default, and what the demo uses). A key only makes the wording more natural.

Q. What does "neuro-symbolic" mean here?

Symbolic = the rule-based CSP engine (exact, explainable). *Neural* = the optional language model (fluent but unreliable). We combine them so each does what it is good at.

Q. What is the difference between forward checking and AC-3?

Forward checking only removes a placed digit from that cell's direct peers. AC-3 keeps going: whenever a domain shrinks, it re-checks the neighbours, until every pair of peers is consistent. AC-3 prunes more but costs more per step.

Q. Why MRV?

Choosing the cell with the fewest candidates (fail-first) finds dead ends early and keeps the search tree small. It also matches how humans play: look at the most constrained cell first.

Q. How do you know a generated puzzle has exactly one solution?

The generator only keeps a blank if the puzzle can still be finished with forced, technique-based deductions. Forced deductions never branch, so if they fill the whole grid there is only one possible solution.

Q. How are puzzle levels decided?

By the hardest technique the puzzle *needs*: beginner = singles only; intermediate = needs pairs, pointing or box/line; advanced = needs a Naked Triple or X-Wing.

Q. What's the difference between "Check square" and "What-If"?

Check square judges a digit against the rules and the solution (Case 1/2/3). What-If plays the move out on a copy of the board and shows the chain of consequences, so you can see *why* it fails.

Q. Can the hints spoil the puzzle?

Only if you ask. The ladder goes nudge → technique → reveal, and the AI Teacher withholds the digit unless the message clearly asks for the answer ("just tell me").

Q. What are the limitations?

Only 9 techniques (no Swordfish, XY-Wing or chains), so the hardest puzzles still need What-If. The verifier checks "cell = digit" claims but not every softer claim about candidates. Sessions live in memory (one board per server).

Q. Where did the old benchmark / comparison part go?

We removed the baseline solver, benchmark scripts and charts to focus the project on the coach. The CSP solver, with all its heuristics, is still in `src/` and still tested. The old material is in the git history (first commit).

Q. I cloned the repo and there is no data/ folder. Is that a problem?

No. `data/` is git-ignored. If `puzzle_bank.json` is missing, the app builds a new bank the first time it is needed. That takes a little while, and your puzzles will differ from ours.

10. What to review before the evaluation

- Run the app and try one lesson, one puzzle, the hint ladder, What-If and a few AI Teacher questions.
- Read `sudoku_ai_coach/README.md` for the full design and the API table.
- Note anything unclear or broken and share it in the team chat.